Introduction to Program Synthesis (SS 2026)

Chapter 1 - Introduction

Dr. rer. nat. Roman Kalkreuth

Chair for AI Methodology (**AIM**), Faculty of Computer Science,
RWTH Aachen University, Germany

Why program synthesis? Some major software bug issues...

- 1962 • Mariner 1 Spacecraft → incorrect guidance signals → $320 million
- 1994 • Pentium FDIV Bug → lookup table bug → $475 million
- 1996 • ESA Ariane 5 Flight V88 → conversion error → $370 million
- 2019 • Boeing 737 Max → MCAS software bug → fatalities
- 2024 • Intel Raptor Lake → stability issues
- 2024 • CrowdStrike Falcon software update → world-wide outage

Why program synthesis?

- Software is **complex** and **fragile**
- Prone to human error
- Automated search and verification $\rightarrow$ Holy grail of software development?
- Well, its complicated[1]
  - $\rightsquigarrow$ Despite the hype, LLMs lack genuine formal reasoning capabilities[2345]
  - $\rightsquigarrow$ A well-established benchmark to test reasoning capabilities is ARC[6]
  - $\rightsquigarrow$ More issues and shortcomings will be addressed later in the course

---

1 https://hackernoon.com/testing-llms-on-solving-leetcode-problems-in-2025

2 https://garymarcus.substack.com/p/llms-dont-do-formal-reasoning-and

3 https://arxiv.org/abs/2410.05229

4 https://garymarcus.substack.com/p/breaking-llm-reasoning-continues

5 https://arxiv.org/pdf/2602.06176

6 https://arcprize.org/arc-agi

General Idea

- Software development process $\rightarrow$ can be automated (to some extend)
- Synthesis of **correct** and **efficient** computer programs with respect to predefined specifications
- Universal approach $\rightarrow$ not limited to a specific programming language, paradigm or, level

Definition and Problem Statement

Definition (Program Synthesis)

Automated discovery of executable programs that match predefined forms of constraints such as input-output relations.

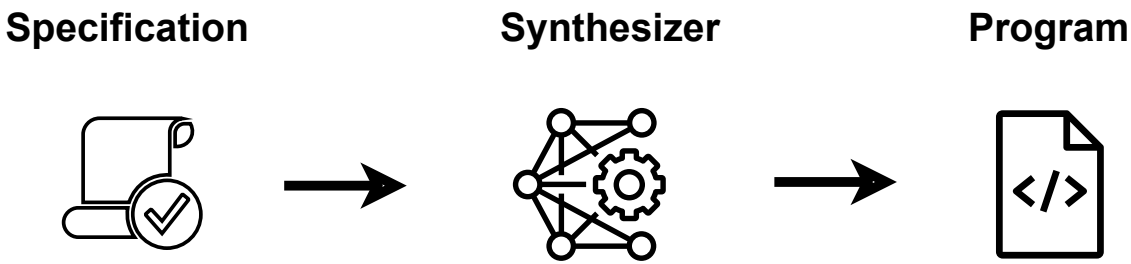
Fundamental Notations

Table: Notation

Symbol	Definition
$\mathcal{P}$	Program
$\mathcal{X}$	Set of inputs
$\mathcal{Y}$	Set of outputs
Φ	Constraints
ψ	Specification

Program Synthesis

- Generation of computer programs from a collection of **artifacts**
  - ⇝ **Automated search** in a space of possible programs
  - ⇝ Matching **semantic** and **syntactic requirements**
- Interaction between **synthesis** and **machine learning**

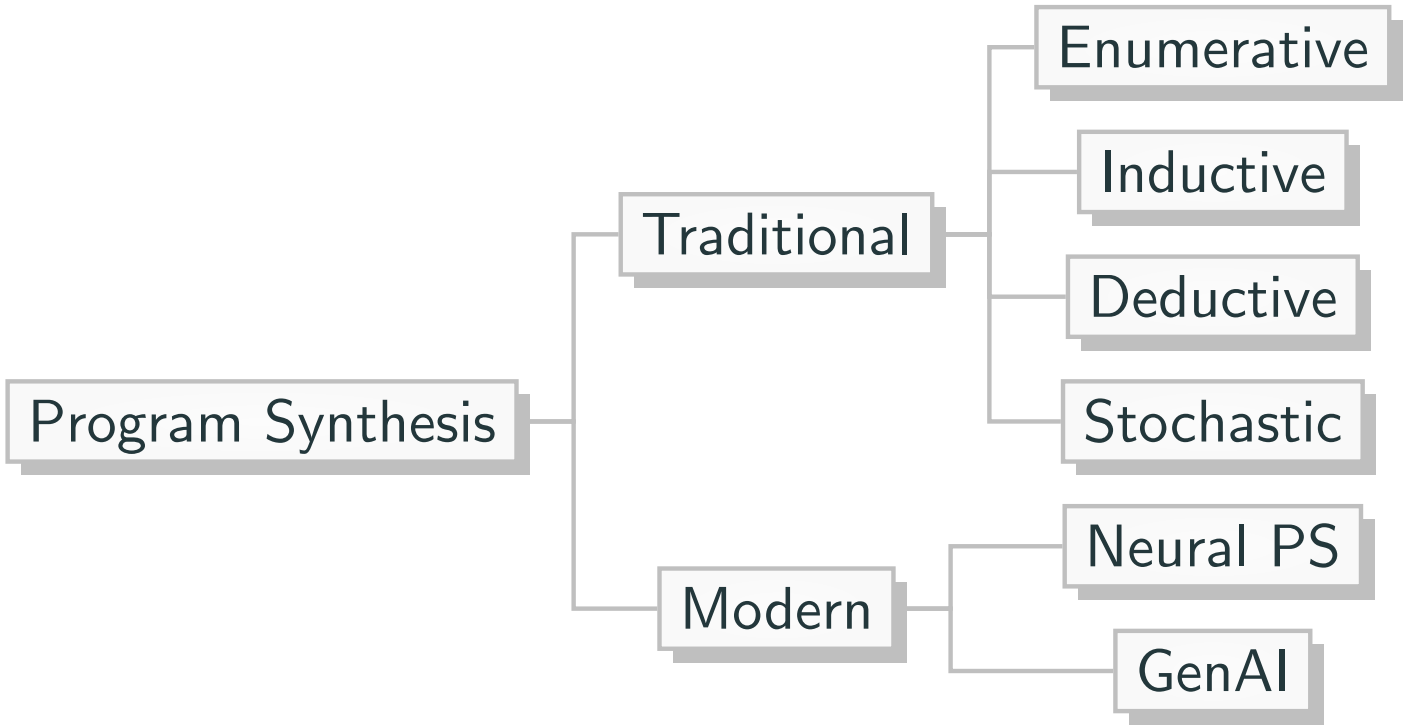


General Definition and Problem Statement

Definition (Program Statement)

Seek a program P that satisfies a specification ψ on a input set $\mathcal{X}$ and is subject to constraints Φ.

Taxonomy (excerpt)



Further Classification

- PS search methods are often performed directly on **high-level symbolic representations** of problems
  - $\rightsquigarrow$ State-of-the-art methods utilise **neuro-symbolic** and **evo-symbolic** search
  - $\rightsquigarrow$ Discovery of human-readable solutions
- Modern PS methodology $\rightarrow$ leveraged with ML paradigms
- PS can be therefore nowadays considered a **symbolic AI/ML** methodology

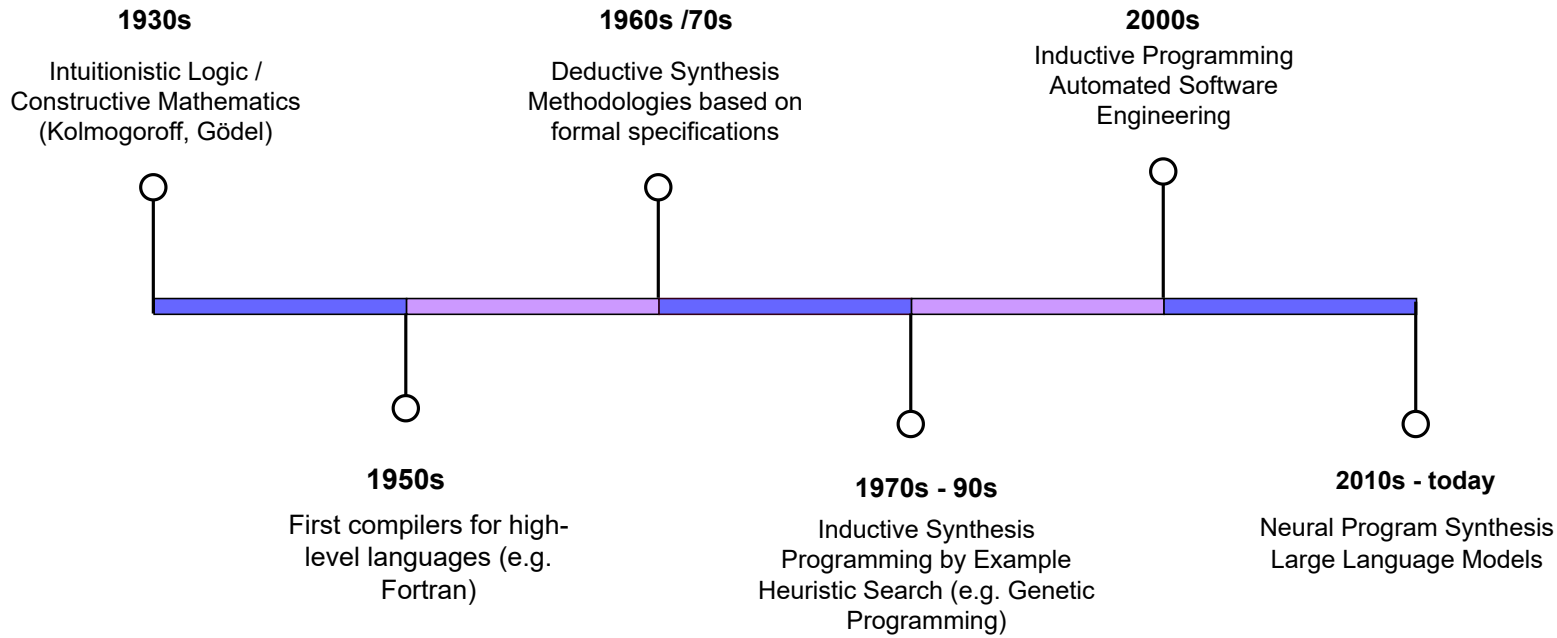
Scope and Distinction

- Synthesis of programs from a collection of given artifacts
  - $\rightsquigarrow$ **Artifacts** $\rightarrow$ represent semantic and syntactic requirements (for the programs)
- Program synthesis by its definition is not (merely):
  - $\rightsquigarrow$ **Compilation** $\rightarrow$ Although compilation and synthesis are very closely related as they share the similar goals
  - $\rightsquigarrow$ **Machine learning** $\rightarrow$ Incremental and transformative synthesis process that is nowadays *backed and guided* by ML concepts and techniques
  - $\rightsquigarrow$ **Optimisation** $\rightarrow$ However, definition of one or more optimisation objectives for the synthesis process is often required

Historical Background

1932	Discovery of algorithms with proofs (Kolmogorov, 1932) [Kol32]
1954	FORTRAN Automatic Coding System (Backus et al., 1954) [Bac+57]
1957	Applications of recursive arithmetic to the problem of circuit synthesis (Alonzo Church) [Fri63]
1969	Solving Sequential Conditions by Finite State Strategies (Buchi and Landweber) [BL90]
1971	Toward automatic program synthesis (Mannar et al., 1971) [MW71]
1975	Transformational synthesis (Manna & Waldinger, 1975) [Kol32]
1977	*Synthesis-by-transformations* paradigm (Burstall & Darlington, 1977) [BJ77]
1979-80	Automated Deduction (Manna & Waldinger, 1979; Manna & Waldinger, 1980; Bibel, 1980) [MW79; MW80; Bib80]
1980s	Efficient strategies for synthesis (generate & test, divide & conquer, problem reduction) (Smith, 1985b; Smith, 1987b) [Dou83; Dou86]
1985	Adaptive Generation of Simple Sequential Programs (Nichael L. Cramer) [Cra85]
1989	Hierarchical Genetic Algorithms Operating on Populations of Computer Programs (John R. Koza) [Koz89]
1992	Genetic Programming (John R. Koza) [Koz93]

Timeline


1930s
Intuitionistic Logic / Constructive Mathematics (Kolmogoroff, Gödel)
1960s /70s
Deductive Synthesis Methodologies based on formal specifications
2000s
Inductive Programming Automated Software Engineering
1950s
First compilers for high-level languages (e.g. Fortran)
1970s - 90s
Inductive Synthesis Programming by Example Heuristic Search (e.g. Genetic Programming)
2010s - today
Neural Program Synthesis Large Language Models

Paradigms

- **Proofs-as-programs:**
  - Synthesis of an algorithm $\rightarrow$ constructive proof of the statement
- **Synthesis by transformations:**
  - Derivation of programs from given specifications by forward propagation
  - Originally based on rewrite rules that encode logical laws

Paradigms

- **Deductive**
  - Deductive reasoning → specific conclusions are drawn from general premises
  - General versions being transformed to a specified version that matches the specification
  - **Top-down approach** → from general information to the specific conclusions
- **Inductive**
  - Inductive reasoning → drawing general conclusions based on specific observations
  - Incremental synthesis based on given examples (i.e. input-output mappings)
  - **Bottom-up approach** → from specific to general
- **Stochastic** → Applying principles of randomised search heuristics
- **Neural** → Use of deep learning based methodologies

Search spaces

- **Symbolic:** Compositions from a finite set of functions $\mathcal{F}$ and set of finite terminals (e.g. variables or constants) $\mathcal{T}$
  - $\mathcal{P} \in \mathcal{F} \times \mathcal{T}$
  - Often represented as non-linear data structures such as trees and graphs
- **Syntactical:** Compositions from nonterminal symbols $\mathcal{N}$ and terminal symbols $\mathcal{T}$ in accordance to production rules $\mathcal{P}$
  - Syntax space is language specific
  - Terminal symbols $\rightarrow$ Functions or operators
  - Nonterminal symbols $\rightarrow$ Variables or constants
- **Semantic:** Discrete or continuous output space of candidate programs
  - $\mathcal{P}(\mathcal{X}) \in \mathbb{R}^n$ or $\mathbb{N}^n$

Objectives

- **Feasibility:** Discovery whether a feasible program can be obtained that matches the predefined specification
  - Usually no constraints specified
  - No prior knowledge given $\rightarrow$ *synthesis-from-scratch*
- **Efficiency:** Consideration of optimisation objectives to obtain a more efficient solution
  - Minimisation of runtime and/or complexity
- **Reliability:** Construction of robust programs that guarantee the predefined specification

Challenges

- Search in symbolic and syntax space is often **ill-conditioned**:
  - Large search spaces
  - Roughness of the cost function landscape
  - No gradient descent $\rightarrow$ gradient-free methods needed
  - Poor local search features
- **Fragility** of syntactical compositions
- Programming requires **formal reasoning**

References I

[Kol32] A. Kolmogoroff. "Zur Deutung der intuitionistischen Logik". In: *Mathematische Zeitschrift* 35.1 (1932), pp. 58–65.

[Bac+57] J. W. Backus et al. "The FORTRAN automatic coding system". In: *Papers Presented at the February 26-28, 1957, Western Joint Computer Conference: Techniques for Reliability*. Association for Computing Machinery, 1957, pp. 188–198.

[Fri63] J. Friedman. "Alonzo Church. Application of recursive arithmetic to the problem of circuit synthesis". In: *Journal of Symbolic Logic* 28.4 (1963), pp. 289–290.

[BL90] J. R. Buchi and L. H. Landweber. "Solving Sequential Conditions by Finite-State Strategies". In: *The Collected Works of J. Richard Büchi*. Springer New York, 1990, pp. 525–541.

[MW71] Z. Manna and R. J. Waldinger. "Toward Automatic Program Synthesis". In: *Communications of the ACM* 14.3 (1971), pp. 151–165.

References II

[BJ77] R. M. Burstall and D. John. "A Transformation System for Developing Recursive Programs". In: *Journal of the ACM* 24.1 (1977), pp. 44–67.

[MW79] Z. Manna and R. Waldinger. "Synthesis: Dreams → Programs". In: *IEEE Transactions on Software Engineering* SE-5.4 (1979), pp. 294–328.

[MW80] Z. Manna and R. Waldinger. "A Deductive Approach to Program Synthesis". In: *ACM Trans. Program. Lang. Syst.* 2.1 (1980), pp. 90–121.

[Bib80] W. Bibel. "Syntax-directed, semantics-supported program synthesis". In: *Artificial Intelligence* 14.3 (1980), pp. 243–261.

[Dou83] R.S. Douglas. "A Problem Reduction Approach to Program Synthesis". In: *Proceedings of the 8th International Joint Conference on Artificial Intelligence. Karlsruhe, FRG, August 1983*. William Kaufmann, 1983, pp. 32–36.

References III

[Dou86] R. S. Douglas. "Top-Down Synthesis of Divide-and-Conquer Algorithms". In: *Readings in Artificial Intelligence and Software Engineering*. Ed. by Charles Rich and Richard C. Waters. Morgan Kaufmann, 1986, pp. 35–61.

[Cra85] N. L. Cramer. "A Representation for the Adaptive Generation of Simple Sequential Programs". In: *Proceedings of the 1st International Conference on Genetic Algorithms*. L. Erlbaum Associates Inc., 1985, pp. 183–187.

[Koz89] J. R. Koza. "Hierarchical Genetic Algorithms Operating on Populations of Computer Programs". In: *Proceedings of the 11th International Joint Conference on Artificial Intelligence. Detroit, MI, USA, August 1989*. Morgan Kaufmann, 1989, pp. 768–774.

[Koz93] J. R. Koza. *Genetic programming - on the programming of computers by means of natural selection*. Complex adaptive systems. MIT Press, 1993. ISBN: 978-0-262-11170-6.